ControlAPI.h File Reference

```
#include <fstream>
#include <cstdint>
```

Go to the source code of this file.

Macros

#define	**API_EXPORT**
#define	**API_EXPORT_CLASS**
#define	**API_EXPORT_CLASS**
#define	**ERROR_CODE_TYPE** bool
#define	**CLA_FN**(name)
#define	**CLA_FNDEF**(name)

Functions

ERROR_CODE_TYPE	**AddErrorMessage** (const std::string &error_message, bool dothrow=true)
API_EXPORT ERROR_CODE_TYPE CLA_FN	**Create** (bool InitializeAfx, bool InitializeAfxSocket) Threading model.
API_EXPORT void **CLA_FN**	**Cleanup** () This function must be called when you are finished using the API. The class wrapped version of this API calls this function in its destructor.
API_EXPORT bool **CLA_FN**	**DidErrorOccur** () Check if an error has occurred since the last call to GetLastError.
API_EXPORT const char ***CLA_FN**	**GetLastError** () returns the last error messages
API_EXPORT void **CLA_FN**	**Configure** (bool _DisplayErrors) Configures if the ControlAPI should display error messages.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**LoadFromJSONFile** (const char *filename) Loads a configuration from a JSON file. Before you can use the control system, you either must read a json configuration file, or define the devices in the API.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AutoConfigure** (const char *filename="") Read the rack auto-configuration, convert it to the standard config schema, and load it.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**Initialize** ()

After Configuring the hardware, e.g. by loading a json configuration file, this function must be called to initialize the hardware.

API_EXPORT void **CLA_FN**	**SwitchDebugMode** (bool OnOff, const char *FileName) Switches FPGA debug mode on. In debug mode, the FPGA sequencers display more information on their USB-UART port, being slowed down a bit by that. To write a human-readable sequence file, pass a filename to SendSequence instead.
API_EXPORT void **CLA_FN**	**TransmitOnlyDifferenceBetweenCommandSequenceIfPossible** (bool OnOff) Transmits only changes of sequence over TCP/IP to sequencer in order to safe time. Once a sequence is in the sequencer's memory, the next sequence is compared to the one in memory and only the changes are transmitted over TCP/IP, if this results in less transmission data.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**IsReady** () Checks if the ControlAPI is ready to be used.
API_EXPORT void **CLA_FN**	**StartAssemblingSequence** () Starts assembling the first sequence in the sequency buffer. Clears any previous sequence. Must be called before adding commands to the sequence.
API_EXPORT void **CLA_FN**	**StartAssemblingNextSequence** () Starts assembling the second or higher sequence in the sequence buffer. Clears any previous sequence. Must be called before adding commands to the sequence.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetRegister** (const unsigned int &Sequencer, const unsigned int &Address, const unsigned int &SubAddress, const uint8_t *Data, const unsigned long &DataLength_in_bit, const uint8_t &StartBit=0) This function sets a register for a device on the sequencer. What this means depends on the device type. This function gives us an easy way to add new functionality to a device without having to programm new DLL functions. SetRegister maps to the register map given in the device's datasheet. Note: this is not currently implemented in the API.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetValue** (const unsigned int &Sequencer, const unsigned int &Address, const unsigned int &SubAddress, const uint8_t *Data, const unsigned long &DataLength_in_bit, const uint8_t &StartBit=0) This function sets a value for a device on the sequencer. What this means depends on the device type. This function gives us an easy way to add new functionality to a device without having to

	programm new DLL functions. In contrast to SetRegister, SetValue can execute more complex operations, such as calculating a DDS frequency tuning word from the given frequency and then programming that. There can be several SetValue functions for the same SetRegister function. There can be SetValue functions that set some parameter, or that trigger some action, not even necessarily related to the registers of the device.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetValueSerialDevice** (const unsigned int &Sequencer, const unsigned int &Address, const unsigned int &SubAddress, const uint8_t *Data, const unsigned long &DataLength_in_bit, const uint8_t &StartBit=0) As SetValue, but for serial devices.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetRegisterSerialDevice** (const unsigned int &Sequencer, const unsigned int &Address, const unsigned int &SubAddress, const uint8_t *Data, const unsigned long &DataLength_in_bit, const uint8_t &StartBit=0) As SetRegister, but for serial devices.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**Wait_ms** (double time_in_ms) Wait for a given time in ms.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetTime_ms** (double &time_in_ms) Get the current time in the currently assembled sequence in ms.
API_EXPORT unsigned int CLA_FN	**GetNumberOfSequencers** () Get the number of configured sequencers.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetTimeOfSequencer_ms** (const unsigned int &Sequencer, double &time_in_ms) Get the current time of a specific sequencer in the currently assembled sequence in ms.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetTimeDebtOfSequencer_ms** (const unsigned int &Sequencer, double &time_debt_in_ms) Get the time debt of a specific sequencer in the currently assembled sequence in ms.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetNextBufferPositionOfMasterSequencer** (unsigned long &next_buffer_position) Get the position of the next empty buffer slot of master sequencer in the currently assembled sequence.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetPeriodicTrigger_ms** (double PeriodicTriggerPeriod_in_ms, double PeriodicTriggerAllowedWaitTime_in_ms) Set the periodic trigger timing used by the master sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetNextCycleNumber** (long &NextCycleNumber) Get next cycle number of master sequencer.

API_EXPORT ERROR_CODE_TYPE CLA_FN	**ResetCycleNumber** () Reset cycle number of master sequencer to 0.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**TransmitI2CPort** (uint8_t I2C_port, uint8_t I2C_destination, uint8_t I2C_address, uint16_t send_length, uint8_t *send_data, uint16_t receive_length, uint8_t *receive_data, uint32_t I2C_clock_frequency_in_Hz, bool &I2C_success, bool fail_silently) Read data from an I2C port.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetPSOptions** (uint8_t options) Set PS option flags on the master sequencer controller.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**WriteConfigEEPROM** (uint8_t SequencerID, uint8_t RackNr, uint8_t SlotNr, const char *data, size_t length) Write configuration data to the EEPROM of a rack slot.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**ReadConfigEEPROM** (uint8_t SequencerID, uint8_t RackNr, uint8_t SlotNr, char *data, size_t &length, bool &I2C_success) Read the complete configuration EEPROM of a rack slot.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**WriteConfigAddress** (uint8_t SequencerID, uint8_t RackNr, uint8_t SlotNr, uint8_t address) Write the configuration address byte of a rack slot.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**ReadConfigAddress** (uint8_t SequencerID, uint8_t RackNr, uint8_t SlotNr, uint8_t &address, bool &I2C_success) Read the configuration address byte of a rack slot.
API_EXPORT const char *CLA_FN	**ReadConfiguration** (const char *filename="") Read the rack auto-configuration and return it as JSON text.
API_EXPORT const char *CLA_FN	**GetAutoConfigJSON** (const char *filename="") Read the rack auto-configuration, convert it to the standard config schema, and return it as JSON text.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**StartAssemblingCPUCommandSequence** () Start assembling a CPU command sequence on the master sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddCPUCommand** (const char *command) Add a CPU command to the currently assembled CPU command sequence on the master sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**ExecuteCPUCommandSequence** (unsigned long ethernet_check_period_in_ms) Execute the assembled CPU command sequence.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**StopCPUCommandSequence** () Stop CPU command sequence at next programmed stop point.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**InterruptCPUCommandSequence** () Interrupt CPU command sequence immediately.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetCPUCommandErrorMessages** ()

	Request CPU command error messages from the master sequencer. The errors will be displayed on the FPGA UART and copied into the API error buffer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**PrintCPUCommandErrorMessages** () print CPU command error messages on FPGA UART.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**PrintCPUCommandSequence** () print CPU command sequence on FPGA UART.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetVoltage** (const unsigned int &Sequencer, const unsigned int &Address, double Voltage) Sets the voltage of an analog output device.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetDigitalOutput** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t BitNr, bool OnOff) Sets a digital output to high or low.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetStartFrequency** (const unsigned int &Sequencer, const unsigned int &Address, double Frequency) Sets the start frequency of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetStopFrequency** (const unsigned int &Sequencer, const unsigned int &Address, double Frequency) Sets the stop frequency of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetModulationFrequency** (const unsigned int &Sequencer, const unsigned int &Address, double Frequency) Sets the modulation frequency of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetPower** (const unsigned int &Sequencer, const unsigned int &Address, double Power) Sets the power of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetAttenuation** (const unsigned int &Sequencer, const unsigned int &Address, double Attenuation) Sets the attenuation of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetIOUpdateEnabled** (const unsigned int &Sequencer, const unsigned int &Address, bool IOUpdateEnabled) Enables or disables automatic IO update after commands sent to a DDS device that supports IO update control.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetStartFrequencyTuningWord** (const unsigned int &Sequencer, const unsigned int &Address, uint64_t FrequencyTuningWord) Sets the start frequency tuning word of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetStopFrequencyTuningWord** (const unsigned int &Sequencer, const unsigned int &Address, uint64_t FrequencyTuningWord)

	Sets the stop frequency tuning word of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetFSKMode** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t mode) Sets the FKS mode of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetRampRateClock** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t rate) Sets the ramp rate clock of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetClearACC1** (const unsigned int &Sequencer, const unsigned int &Address, bool OnOff) Clears the ACC1 bit of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetTriangleBit** (const unsigned int &Sequencer, const unsigned int &Address, bool OnOff) Sets the triangle bit of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetFSKBit** (const unsigned int &Sequencer, const unsigned int &Address, bool OnOff) Sets the FSK bit of a DDS (for now a AD9854 DDS).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetFrequency** (const unsigned int &Sequencer, const unsigned int &Address, double Frequency) Sets the frequency of a DDS.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetFrequencyTuningWord** (const unsigned int &Sequencer, const unsigned int &Address, uint64_t FrequencyTuningWord) Sets the frequency tuning word of a DDS.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**Reset** (const unsigned int &Sequencer, const unsigned int &Address) Resets the AD9959.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetFrequencyOfChannel** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t channel, double Frequency) Sets the frequency of a multi channel DDS (for now the AD9959).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetFrequencyTuningWordOfChannel** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t channel, uint64_t FrequencyTuningWord) Sets the frequency tuning word of a multi channel DDS (for now the AD9959).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetPhaseOfChannel** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t channel, double Phase) Sets the phase of a multi channel DDS (for now the AD9959).
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetPowerOfChannel** (const unsigned int &Sequencer, const unsigned int &Address, uint8_t channel, double Power) Sets the power of a multi channel DDS (for now the AD9959).

API_EXPORT void CLA_FN	**SendSequence** (const char *FileName="") Send the sequence that was previously assembled to FPGA SoM, but do not execute it.
API_EXPORT void CLA_FN	**ExecuteSequence** (const char *FileName="") Send the sequence that was previously assembled to FPGA and execute it.
API_EXPORT void CLA_FN	**RepeatSequence** () Execute the sequence that was sent to the FPGA.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**WaitTillFinished** (double timeout_in_s=0) Waits until the sequence is finished.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**GetSequenceExecutionStatus** (bool &running, unsigned long long &DataPointsWritten) Get the current status of the sequence execution.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**WaitTillEndOfSequenceThenGetInputData** (uint8_t *&buffer, unsigned long &buffer_length, unsigned long &EndTimeOfCycle, double timeout_in_s) Waits until the sequence is finished, then gets the input data from the FPGA sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetTimeDebtGuard_in_ms** (const double &MaxTimeDebt_in_ms) Sets a guard time. If sequencer commands make the time advance more than the guard time beyond what's allowed by Wait_ms, an error will be recorded (check with **DidErrorOccur()** if that happened) or thrown.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerStartAnalogInAcquisition** (const unsigned int &Sequencer, const uint8_t &analog_in_type, const uint8_t &SPI_CS, const uint8_t &ChannelNumber, const uint32_t &NumberOfDataPoints, const double &DelayBetweenDataPoints_in_ms) Start analog acquisition on the specified channel. This function places a commandto for the FPGA in the sequencer buffer. The analog in acquisition will start when this command is executed by the FPGA. Only one analog in acquisition can hapen at each moment. A new one can start right after the previous one is finished. The data will be returned at the end of a sequence when using CLA_WaitTillEndOfSequenceThenGetInputData.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerWriteInputMemory** (const unsigned int &Sequencer, unsigned long input_buf_mem_data, bool write_next_address=1, unsigned long input_buf_mem_address=0) Writes a value to the input memory of the sequencer. This is useful to mark the start or end of a data acquisition, or to mark the type of experimental run in the full fledged version of the ControlAPI, which can cycle sequences automatically in the

	background. To execute this function, a command for the FPGA must be placed in the sequencer buffer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerCalcAD9854FrequencyTuningWord** (const unsigned int &Sequencer, uint64_t ftw0, uint8_t bit_shift=22) To use a DDS like a VCO: calculates the frequency tuning word for a AD9854 DDS using a centre frequency and a detuning proportional to a ADC value. The frequency tuning word is proportional to the period of the DDS output frequency, i.e. it's 1/frequency As with all Analog Devices DDS devices, the value of the frequency tuning word is determined by FTW = (Desired Output Frequency × 2^N)/SYSCLK where: N is the phase accumulator resolution (48 bits in this instance). Desired Output Frequency is expressed in hertz. FTW (frequency tuning word) is a decimal number. The desired frequency is f = f0 + c * voltage = f0 + deltaf voltage is the 16-bit value provided by the ADC We don't want to use a multiplication. Instead we approximate, using epsilon = deltaf/f0 << 1. delta f = c* voltage ftw = 1/f = 1/(f0 + deltaf) = 1/ (f0 * (1 + deltaf/f0)) = (1/f0)*(1/(1+epsilon)) ~ ftw0 * (1-epsilon) = ftw0 - ftw0*deltaf/f0 = ftw0 - ftw0 * ftw0 * c * voltage = ftw0 - scale * ADC_value We replace the multiplication by a bitshift, i.e. we allow scale = 2^n with n=[0...32].
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerWriteSystemTimeToInputMemory** (const unsigned int &Sequencer) Writes the current FPGA clock ticks to the input memory of the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerSwitchDebugLED** (const unsigned int &Sequencer, unsigned int OnOff) Switches the debug LED of the FPGA on or off.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetSequencerDigitalOut** (const unsigned int &Sequencer, uint8_t dig_out_pattern) Sets the sequencer digital output pattern.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SetSequencer_PL_to_PS_command** (const unsigned int &Sequencer, uint8_t PL_to_PS_command) Sets the sequencer PL-to-PS command byte.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SwitchSequencerBuzzer** (const unsigned int &Sequencer, bool OnOff) Switches the sequencer buzzer on or off.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerIgnoreTCPIP** (const unsigned int &Sequencer, bool OnOff) Switches the sequencer to ignore TCP/IP commands. This is useful to prevent the sequencer from being interrupted by TCP/IP commands while executing a timing critical task, such as

	transferring input data from the FPGA BRAM to the DDR. This can be useful if lots of data is acquired at a high rate. Lot's of meaning: more than the size of the BRAM input buffer. How much that is depends how you configured the Vivado project that generates the FPGA sequencer firmware.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**UseEdgeTriggeredLatches** (const unsigned int &Sequencer, bool UseEdgeTriggeredLatches) Configures the sequencer to use edge-triggered latches.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerAddMarker** (const unsigned int &Sequencer, unsigned char marker) Places a marker in the sequencer buffer. The FPGA SOM's CPU will see this marker and can react to it, e.g. write it to the USB-UART for debugging.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerSetTimeDebtGuard_in_ms** (const unsigned int &Sequencer, const double &MaxTimeDebt_in_ms) Sets the time debt guard for a specific sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerSetLoopCount** (const unsigned int &Sequencer, unsigned int loop_count) Sets the loop count for a sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerJumpBackward** (const unsigned int &Sequencer, unsigned int jump_length, bool unconditional_jump=true, bool condition_0=false, bool condition_1=false, bool condition_PS=false, bool condition_dig_in=false, uint8_t dig_in_bit_nr=0, bool loop_count_greater_zero=false) Jumps backward in the sequence.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerJumpForward** (const unsigned int &Sequencer, unsigned int jump_length, bool unconditional_jump=true, bool condition_0=false, bool condition_1=false, bool condition_PS=false, bool condition_dig_in=false, uint8_t dig_in_bit_nr=0) Jumps forward in the sequence.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerTransmitI2C** (const unsigned int &Sequencer, uint8_t I2C_port, uint8_t I2C_length_out, uint8_t I2C_length_in, uint8_t *data_out) Writes an I2C command to the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerTransmitSPI** (const unsigned int &Sequencer, uint8_t chip_select, uint16_t number_of_bits_out, const uint8_t *data_out, uint8_t number_of_bits_in, bool start_now) Writes an SPI transmit command to the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerRepeatedOutIn** (const unsigned int &Sequencer, uint16_t number_of_datapoints, double

	delay_between_datapoints_in_ms, uint8_t RepeatedOutInCommand) Configures repeated input/output operations in the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerSetSPITiming** (const unsigned int &Sequencer, uint16_t SPI_delay_CS_low_start_wait, uint16_t SPI_delay_write, uint16_t SPI_delay_pause_before_read, uint16_t SPI_delay_read, uint16_t SPI_delay_CS_low_end_wait) Loads SPI timing parameters into the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerSetSPIMode** (const unsigned int &Sequencer, uint8_t SPI_mode) Sets the SPI mode for the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SequencerSetI2CParameters** (const unsigned int &Sequencer, uint8_t I2C_0_Destination, uint8_t I2C_delay_start_stop, uint8_t I2C_delay_data_setup, uint8_t I2C_delay_clock_high, uint8_t I2C_delay_clock_low, uint8_t I2C_delay_pause_before_read) Sets I2C parameters for the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**SelectRackSlot** (const unsigned int &Sequencer, uint8_t rack_nr, uint8_t slot_nr) Selects a slot in one of the racks connected in a chain to a sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**ResetI2CMultiplexer** (const unsigned int &Sequencer) Resets the I2C multiplexer used for rack-slot selection on a sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceSequencer** (unsigned int id, const char *type, const char *ip, unsigned int port, bool master, unsigned int startDelay, double clockFrequency, unsigned long FPGAClockToBusClockRatio, bool useExternalClock, bool useStrobeGenerator, bool **UseEdgeTriggeredLatches**, bool connect) Add a device sequencer to the list.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceAnalogOut16bit** (unsigned int sequencer, unsigned int startAddress, unsigned int numberChannels, bool signedValue, double minVoltage, double maxVoltage) Add a 16 bit analog output device to the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceDigitalOut** (unsigned int sequencer, unsigned int address, unsigned int numberChannels) Add a digital output device to the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceAD9854** (unsigned int sequencer, unsigned int address, unsigned int version, double externalClockFrequency, uint8_t PLLReferenceMultiplier, unsigned int frequencyMultiplier) Add a AD9854 device to the sequencer.

API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceAD9858** (unsigned int sequencer, unsigned int address, double externalClockFrequency, unsigned int frequencyMultiplier) Add a AD9858 device to the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceAD9959** (unsigned int sequencer, unsigned int address, double externalClockFrequency, unsigned int frequencyMultiplier, bool AD9958) Add a AD9959 device to the sequencer.
API_EXPORT ERROR_CODE_TYPE CLA_FN	**AddDeviceAnalogIn12bit** (unsigned int sequencer, unsigned int chipSelect, bool signedValue, double minVoltage, double maxVoltage) Add a 12 bit analog input device to the sequencer.

Variables

constexpr int	**MaxLastError** = 10
std::string	**LastError** [**MaxLastError**]
int	**LastErrorIndex**
bool	**ErrorListWrapAround**

Macro Definition Documentation

◆ API_EXPORT

#define API_EXPORT

◆ API_EXPORT_CLASS [1/2]

#define API_EXPORT_CLASS

◆ API_EXPORT_CLASS [2/2]

#define API_EXPORT_CLASS

◆ CLA_FN

#define CLA_FN (name)

Value:

```
CLA_##name
```

◆ CLA_FNDEF

#define CLA_FNDEF (name)

Value:

```
CLA_##name
```

◆ ERROR_CODE_TYPE

#define ERROR_CODE_TYPE bool

Function Documentation

◆ AddCPUCommand()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddCPUCommand (const char * command)

Add a CPU command to the currently assembled CPU command sequence on the master sequencer.

Parameters

command string.

Returns

See return convention above.

◆ AddDeviceAD9854()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddDeviceAD9854 (unsigned int sequencer,
unsigned int address,
unsigned int version,
double externalClockFrequency,
uint8_t PLLReferenceMultiplier,
unsigned int frequencyMultiplier)

Add a AD9854 device to the sequencer.

Parameters

sequencer	the sequencer to use.
address	the address of the device to add.
version	the version of the device to add.
externalClockFrequency	the external clock frequency of the device to add.
PLLReferenceMultiplier	the PLL reference multiplier of the device to add.
frequencyMultiplier	the frequency multiplier of the device to add.

Returns

See return convention above.

◆ AddDeviceAD9858()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddDeviceAD9858 (unsigned int sequencer,
unsigned int address,
double externalClockFrequency,
unsigned int frequencyMultiplier)

Add a AD9858 device to the sequencer.

Parameters

sequencer	the sequencer to use.
address	the address of the device to add.
externalClockFrequency	the external clock frequency of the device to add.
frequencyMultiplier	the frequency multiplier of the device to add.

Returns

See return convention above.

◆ AddDeviceAD9959()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddDeviceAD9959 (unsigned int sequencer,
unsigned int address,
double externalClockFrequency,
unsigned int frequencyMultiplier,
bool AD9958)

Add a AD9959 device to the sequencer.

Parameters

sequencer	the sequencer to use.
address	the address of the device to add.
externalClockFrequency	the external clock frequency of the device to add.
frequencyMultiplier	the frequency multiplier of the device to add.
AD9958	true if the connected device is the two-channel AD9958 variant, false for the four-channel AD9959.

Returns

See return convention above.

◆ AddDeviceAnalogIn12bit()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddDeviceAnalogIn12bit (unsigned int sequencer,
unsigned int chipSelect,
bool signedValue,
double minVoltage,
double maxVoltage)

Add a 12 bit analog input device to the sequencer.

Parameters

sequencer	the sequencer to use.
chipSelect	the chip select of the device to add.
signedValue	true if the device is signed, false if it is unsigned.
minVoltage	the minimum voltage of the device to add.
maxVoltage	the maximum voltage of the device to add.

Returns

See return convention above.

◆ AddDeviceAnalogOut16bit()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddDeviceAnalogOut16bit (unsigned int sequencer,
unsigned int startAddress,
unsigned int numberChannels,
bool signedValue,
double minVoltage,
double maxVoltage)

Add a 16 bit analog output device to the sequencer.

Parameters

sequencer	the sequencer to use.
startAddress	the start address of the device to add.
numberChannels	the number of channels of the device to add.
signedValue	true if the device is signed, false if it is unsigned.
minVoltage	the minimum voltage of the device to add.
maxVoltage	the maximum voltage of the device to add.

Returns

See return convention above.

◆ AddDeviceDigitalOut()

API_EXPORT ERROR_CODE_TYPE CLA_FN AddDeviceDigitalOut (unsigned int sequencer,
unsigned int address,
unsigned int numberChannels)

Add a digital output device to the sequencer.

Parameters

sequencer	the sequencer to use.
address	the address of the device to add.
numberChannels	the number of channels of the device to add.

Returns

See return convention above.

◆ AddDeviceSequencer()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
AddDeviceSequencer ( unsigned int   id,
                     const char *   type,
                     const char *   ip,
                     unsigned int   port,
                     bool           master,
                     unsigned int   startDelay,
                     double         clockFrequency,
                     unsigned long  FPGAClockToBusClockRatio,
                     bool           useExternalClock,
                     bool           useStrobeGenerator,
                     bool           UseEdgeTriggeredLatches,
                     bool           connect )
```

Add a device sequencer to the list.

Parameters

id	the id of the device sequencer to add (this is the number that all other commands are using when sending data to a device on this sequencer).
type	the type of the device sequencer to add.
ip	the ip address of the device sequencer to add.
port	the port of the device sequencer to add.
master	true if the device sequencer is a master, false if it is a slave.
startDelay	the start delay of the device sequencer to add.
clockFrequency	the clock frequency of the device sequencer to add.
FPGAClockToBusClockRatio	the FPGA clock to bus clock ratio of the device sequencer to add.
useExternalClock	true if the device sequencer should use an external clock, false if it should use the internal clock.
useStrobeGenerator	true if the device sequencer should use a strobe generator, false if it should not.
UseEdgeTriggeredLatches	true if the sequencer should use edge-triggered latches, false for the default latch timing.
connect	true if the device sequencer should connect to the device, false if it should not.

Returns

See return convention above.

◆ AddErrorMessage()

ERROR_CODE_TYPE AddErrorMessage (const std::string & error_message,
bool dothrow = true)

◆ AutoConfigure()

API_EXPORT ERROR_CODE_TYPE CLA_FN AutoConfigure (const char * filename = "")

Read the rack auto-configuration, convert it to the standard config schema, and load it.

Parameters

filename Optional base filename used for the generated output files. Pass an empty string to avoid writing files.

Returns

See return convention above.

◆ Cleanup()

API_EXPORT void **CLA_FN** Cleanup ( )

This function must be called when you are finished using the API. The class wrapped version of this API calls this function in its destructor.

Returns

See return convention above.

◆ Configure()

API_EXPORT void **CLA_FN** Configure (bool _DisplayErrors)

Configures if the ControlAPI should display error messages.

Parameters

_DisplayErrors true to display error messages, false to suppress them.

Returns

See return convention above.

◆ Create()

API_EXPORT ERROR_CODE_TYPE CLA_FN Create (bool InitializeAfx,
bool InitializeAfxSocket)

Threading model.

The ControlLight API is intended to be used from one client thread at a time. Concurrent calls into the same API instance are unsupported, including calls made while another thread is waiting for sequence completion or communicating with the FPGA. Client code must serialize API access externally. The API may report an error or behave unpredictably if it is entered concurrently.

Return convention.

Functions returning ERROR_CODE_TYPE return true on success and false on failure in the C API build. In the Python/C++ exception build, ERROR_CODE_TYPE is void and failures are reported by throwing CLA_Exception. Functions returning void perform the requested action and report only through the API error state where noted. Functions returning const char* return pointers owned by the API; callers must copy the string if they need it after the next API call.

Initializes the ControlAPI. If you use a bare function C API, then this function must be called before using any other functions in the API. If you didn't load MFC yet (which is the case if you use Qt), then set

Parameters

InitializeAfx	to true to initialize MFC core
InitializeAfxSocket	to true to initialize MFC socket layer The class wrapped version of this API calls this function in its constructor.

Returns

See return convention above.

◆ DidErrorOccur()

API_EXPORT bool **CLA_FN** DidErrorOccur ( )

Check if an error has occurred since the last call to GetLastError.

Returns

true if an error has occurred since the last call to GetLastError

◆ ExecuteCPUCommandSequence()

API_EXPORT ERROR_CODE_TYPE CLA_FN
ExecuteCPUCommandSequence (unsigned long ethernet_check_period_in_ms)

Execute the assembled CPU command sequence.

Parameters

ethernet_check_period_in_ms period in ms between Ethernet status checks while the CPU command sequence is running. Use 0 to not check the Ethernet status (increases timing precision and speed slightly).

Returns

See return convention above.

◆ ExecuteSequence()

API_EXPORT void **CLA_FN** ExecuteSequence (const char * FileName = "")

Send the sequence that was previously assembled to FPGA and execute it.

Parameters

FileName optional base filename for writing a human-readable sequence debug file. Pass an empty string to disable file writing.

Returns

See return convention above.

◆ GetAutoConfigJSON()

API_EXPORT const char ***CLA_FN** GetAutoConfigJSON (const char * filename = "")

Read the rack auto-configuration, convert it to the standard config schema, and return it as JSON text.

Parameters

filename Optional base filename used for the generated output files. Pass an empty string to avoid writing files.

Returns

JSON text containing the generated configuration.

◆ GetCPUCommandErrorMessages()

API_EXPORT ERROR_CODE_TYPE CLA_FN GetCPUCommandErrorMessages ( )

Request CPU command error messages from the master sequencer. The errors will be displayed on the FPGA UART and copied into the API error buffer.

Returns

See return convention above.

◆ GetLastError()

API_EXPORT const char ***CLA_FN** GetLastError ( )

returns the last error messages

Returns

See return convention above.

◆ GetNextBufferPositionOfMasterSequencer()

API_EXPORT ERROR_CODE_TYPE CLA_FN
GetNextBufferPositionOfMasterSequencer (unsigned long & next_buffer_position)

Get the position of the next empty buffer slot of master sequencer in the currently assembled sequence.

Parameters

next_buffer_position output parameter receiving the next writable command-buffer slot as unsigned long / uint32_t.

Returns

See return convention above.

◆ GetNextCycleNumber()

API_EXPORT ERROR_CODE_TYPE CLA_FN GetNextCycleNumber (long & NextCycleNumber)

Get next cycle number of master sequencer.

Parameters

NextCycleNumber the next cycle number.

Returns

See return convention above.

◆ GetNumberOfSequencers()

API_EXPORT unsigned int **CLA_FN** GetNumberOfSequencers ( )

Get the number of configured sequencers.

Returns

The current number of sequencers known to the API.

◆ GetSequenceExecutionStatus()

API_EXPORT ERROR_CODE_TYPE CLA_FN
GetSequenceExecutionStatus (bool & running,
unsigned long long & DataPointsWritten)

Get the current status of the sequence execution.

Parameters

running returns true if the sequence is running, false if it is not.

DataPointsWritten returns number of data points that were already written out by the FPGA sequencer(s).

◆ GetTime_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN GetTime_ms (double & time_in_ms)

Get the current time in the currently assembled sequence in ms.

Parameters

time_in_ms the current time in ms.

Returns

See return convention above.

◆ GetTimeDebtOfSequencer_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN
GetTimeDebtOfSequencer_ms (const unsigned int & Sequencer,
double & time_debt_in_ms)

Get the time debt of a specific sequencer in the currently assembled sequence in ms.

Parameters

Sequencer the sequencer to use.

time_debt_in_ms output parameter receiving the amount by which this sequencer is ahead of the master sequence time, in ms.

Returns

See return convention above.

◆ GetTimeOfSequencer_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN GetTimeOfSequencer_ms (const unsigned int & Sequencer,
double & time_in_ms)

Get the current time of a specific sequencer in the currently assembled sequence in ms.

Parameters

Sequencer the sequencer to use.

time_in_ms the current time in ms.

Returns

See return convention above.

◆ Initialize()

API_EXPORT ERROR_CODE_TYPE CLA_FN Initialize ( )

After Configuring the hardware, e.g. by loading a json configuration file, this function must be called to initialize the hardware.

Returns

See return convention above.

◆ InterruptCPUCommandSequence()

API_EXPORT ERROR_CODE_TYPE CLA_FN InterruptCPUCommandSequence ( )

Interrupt CPU command sequence immediately.

Returns

See return convention above.

◆ IsReady()

API_EXPORT ERROR_CODE_TYPE CLA_FN IsReady ( )

Checks if the ControlAPI is ready to be used.

Parameters

IsReady true if the ControlAPI is ready to be used, false otherwise.

◆ LoadFromJSONFile()

API_EXPORT ERROR_CODE_TYPE CLA_FN LoadFromJSONFile (const char * filename)

Loads a configuration from a JSON file. Before you can use the control system, you either must read a json configuration file, or define the devices in the API.

Parameters

filename the name of the json file to load.

Loads a configuration from a JSON file. Before you can use the control system, you either must read a json configuration file, or define the devices in the API.

Parameters

filename the name of the file to load the configuration from.

Returns

◆ PrintCPUCommandErrorMessages()

API_EXPORT ERROR_CODE_TYPE CLA_FN PrintCPUCommandErrorMessages ( )

print CPU command error messages on FPGA UART.

Returns

See return convention above.

◆ PrintCPUCommandSequence()

API_EXPORT ERROR_CODE_TYPE CLA_FN PrintCPUCommandSequence ( )

print CPU command sequence on FPGA UART.

Returns

See return convention above.

◆ ReadConfigAddress()

```
API_EXPORT ERROR_CODE_TYPE CLA_FN ReadConfigAddress ( uint8_t    SequencerID,
                                                      uint8_t    RackNr,
                                                      uint8_t    SlotNr,
                                                      uint8_t &  address,
                                                      bool &     I2C_success )
```

Read the configuration address byte of a rack slot.

Parameters

SequencerID	the sequencer to use.
RackNr	the rack number.
SlotNr	the slot number within the rack.
address	output byte receiving the current address value.
I2C_success	output flag indicating whether the address read transaction succeeded.

Returns

See return convention above.

◆ ReadConfigEEPROM()

```
API_EXPORT ERROR_CODE_TYPE CLA_FN ReadConfigEEPROM ( uint8_t    SequencerID,
                                                     uint8_t    RackNr,
                                                     uint8_t    SlotNr,
                                                     char *     data,
                                                     size_t &   length,
                                                     bool &     I2C_success )
```

Read the complete configuration EEPROM of a rack slot.

Parameters

SequencerID	the sequencer to use.
RackNr	the rack number.
SlotNr	the slot number within the rack.
data	output buffer receiving the EEPROM contents.
length	input: available buffer size in bytes, output: number of bytes read.
I2C_success	output flag indicating whether the EEPROM read transaction succeeded.

Returns

See return convention above.

◆ ReadConfiguration()

API_EXPORT const char ***CLA_FN** ReadConfiguration (const char * filename = "")

Read the rack auto-configuration and return it as JSON text.

Parameters

filename Optional output filename. Pass an empty string to avoid writing a file.

Returns

JSON text containing the discovered configuration.

◆ RepeatSequence()

API_EXPORT void **CLA_FN** RepeatSequence ( )

Execute the sequence that was sent to the FPGA.

Returns

See return convention above.

◆ Reset()

API_EXPORT ERROR_CODE_TYPE CLA_FN Reset (const unsigned int & Sequencer,
const unsigned int & Address)

Resets the AD9959.

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the frequency for.

Returns

See return convention above.

◆ ResetCycleNumber()

API_EXPORT ERROR_CODE_TYPE CLA_FN ResetCycleNumber ( )

Reset cycle number of master sequencer to 0.

Returns

See return convention above.

◆ ResetI2CMultiplexer()

API_EXPORT ERROR_CODE_TYPE CLA_FN ResetI2CMultiplexer (const unsigned int & Sequencer)

Resets the I2C multiplexer used for rack-slot selection on a sequencer.

Parameters

Sequencer the sequencer to use.

◆ SelectRackSlot()

API_EXPORT ERROR_CODE_TYPE CLA_FN SelectRackSlot (const unsigned int & Sequencer,
uint8_t rack_nr,
uint8_t slot_nr)

Selects a slot in one of the racks connected in a chain to a sequencer.

Parameters

Sequencer the sequencer to use.

rack_nr rack number in the rack chain.

slot_nr slot number within the selected rack.

◆ SendSequence()

API_EXPORT void **CLA_FN** SendSequence (const char * FileName = "")

Send the sequence that was previously assembled to FPGA SoM, but do not execute it.

Parameters

FileName optional base filename for writing a human-readable sequence debug file. Pass an empty string to disable file writing.

Returns

See return convention above.

◆ SequencerAddMarker()

API_EXPORT ERROR_CODE_TYPE CLA_FN SequencerAddMarker (const unsigned int & Sequencer, unsigned char marker)

Places a marker in the sequencer buffer. The FPGA SOM's CPU will see this marker and can react to it, e.g. write it to the USB-UART for debugging.

Parameters

Sequencer the sequencer to use.

marker marker byte to insert into the sequencer command stream.

Returns

See return convention above.

◆ SequencerCalcAD9854FrequencyTuningWord()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SequencerCalcAD9854FrequencyTuningWord (const unsigned int & Sequencer,
uint64_t ftw0,
uint8_t bit_shift = 22)

To use a DDS like a VCO: calculates the frequency tuning word for a AD9854 DDS using a centre frequency and a detuning proportional to a ADC value. The frequency tuning word is proportional to the period of the DDS output frequency, i.e. it's 1/frequency As with all Analog Devices DDS devices, the value of the frequency tuning word is determined by FTW = (Desired Output Frequency × 2^N)/SYSCLK where: N is the phase accumulator resolution (48 bits in this instance). Desired Output Frequency is expressed in hertz. FTW (frequency tuning word) is a decimal number. The desired frequency is f = f0 + c * voltage = f0 + deltaf voltage is the 16-bit value provided by the ADC We don't want to use a multiplication. Instead we approximate, using epsilon = deltaf/f0 << 1. delta f = c* voltage ftw = 1/f = 1/(f0 + deltaf) = 1/ (f0 * (1 + deltaf/f0)) = (1/f0)*(1/(1+epsilon)) ~ ftw0 * (1-epsilon) = ftw0 - ftw0*deltaf/f0 = ftw0 - ftw0 * ftw0 * c * voltage = ftw0 - scale * ADC_value We replace the multiplication by a bitshift, i.e. we allow scale = 2^n with n=[0...32].

Assuming the ADC provides 16 bits min frequency change: (1 << bit_shift) SYSCLK / (2<<48) frequency range: (1 << (bit_shift+16)) SYSCLK / (2<<48) 2^48= 281,474,976,710,656 SYSCLCK 80000000 bitshift 1<<bitshift deltaf_min deltaf_max 0 1 2.84217E-07 0.018626451 1 2 5.68434E-07 0.037252903 2 4 1.13687E-06 0.074505806 3 8 2.27374E-06 0.149011612 4 16 4.54747E-06 0.298023224 5 32 9.09495E-06 0.596046448 6 64 1.81899E-05 1.192092896 7 128 3.63798E-05 2.384185791 8 256 7.27596E-05 4.768371582 9 512 0.000145519 9.536743164 10 1024 0.000291038 19.07348633 11 2048 0.000582077 38.14697266 12 4096 0.001164153 76.29394531 13 8192 0.002328306 152.5878906 14 16384 0.004656613 305.1757813 15 32768 0.009313226 610.3515625 16 65536 0.018626451 1220.703125 17 131072 0.037252903 2441.40625 18 262144 0.074505806 4882.8125 19 524288 0.149011612 9765.625 20 1048576 0.298023224 19531.25 21 2097152 0.596046448 39062.5 22 4194304 1.192092896 78125 23 8388608 2.384185791 156250 24 16777216 4.768371582 312500 25 33554432 9.536743164 625000 26 67108864 19.07348633 1250000 27 134217728 38.14697266 2500000 28 268435456 76.29394531 5000000 29 536870912 152.5878906 10000000 30 1073741824 305.1757813 20000000 31 2147483648 610.3515625 40000000

Parameters

Sequencer the sequencer to use.

ftw0 the centre frequency tuning word.

bit_shift the bit shift to use. Default is 22, which corresponds to a maximum tuning range of 78kHz and a change of 1.19Hz per ADC value increment.

Returns

See return convention above.

◆ SequencerIgnoreTCPIP()

API_EXPORT ERROR_CODE_TYPE CLA_FN SequencerIgnoreTCPIP (const unsigned int & Sequencer, bool OnOff)

Switches the sequencer to ignore TCP/IP commands. This is useful to prevent the sequencer from being interrupted by TCP/IP commands while executing a timing critical task, such as transferring input data from the FPGA BRAM to the DDR. This can be useful if lots of data is acquired at a high rate. Lot's of meaning: more than the size of the BRAM input buffer. How much that is depends how you configured the Vivado project that generates the FPGA sequencer firmware.

Parameters

Sequencer the sequencer to use.

OnOff true to make the sequencer ignore TCP/IP commands, false to resume normal TCP/IP command handling.

Returns

See return convention above.

◆ SequencerJumpBackward()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
SequencerJumpBackward ( const unsigned int & Sequencer,
    unsigned int jump_length,
    bool unconditional_jump = true,
    bool condition_0 = false,
    bool condition_1 = false,
    bool condition_PS = false,
    bool condition_dig_in = false,
    uint8_t dig_in_bit_nr = 0,
    bool loop_count_greater_zero = false )
```

Jumps backward in the sequence.

Parameters

Sequencer	the sequencer to use.
jump_length	the length of the jump in commands.
unconditional_jump	true to jump unconditionally, false to jump only if a condition is met.
condition_0	true if condition 0 is met, false otherwise.
condition_1	true if condition 1 is met, false otherwise.
condition_PS	true if the PS condition is met, false otherwise.
condition_dig_in	true if the digital input condition is enabled, false otherwise.
dig_in_bit_nr	the digital input bit number to test.
loop_count_greater_zero	true if the loop count is greater than zero, false otherwise.

◆ SequencerJumpForward()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
SequencerJumpForward ( const unsigned int & Sequencer,
                       unsigned int   jump_length,
                       bool           unconditional_jump = true,
                       bool           condition_0 = false,
                       bool           condition_1 = false,
                       bool           condition_PS = false,
                       bool           condition_dig_in = false,
                       uint8_t        dig_in_bit_nr = 0 )
```

Jumps forward in the sequence.

Parameters

Sequencer	the sequencer to use.
jump_length	the length of the jump in commands.
unconditional_jump	true to jump unconditionally, false to jump only if a condition is met.
condition_0	true if condition 0 is met, false otherwise.
condition_1	true if condition 1 is met, false otherwise.
condition_PS	true if the PS condition is met, false otherwise.
condition_dig_in	true if the digital input condition is enabled, false otherwise.
dig_in_bit_nr	the digital input bit number to test.

◆ SequencerRepeatedOutIn()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SequencerRepeatedOutIn (const unsigned int & Sequencer,
uint16_t number_of_datapoints,
double delay_between_datapoints_in_ms,
uint8_t RepeatedOutInCommand)

Configures repeated input/output operations in the sequencer.

Parameters

Sequencer	the sequencer to use.
number_of_datapoints	the number of data points to read/write.
delay_between_datapoints_in_ms	the delay between data points in ms
RepeatedOutInCommand	the command to execute for each data point. 0: stop; 1: repeated SPI transfer; 2: repeated digital in; 3: digital in event tagger for 3: if dig in changes, safes dig in on input memory bit 0:7, bit 8: counter overflow, bit 9: 4-entry fifo overflow, bit 10:31: clock cycle counter; runs till stopped by setting RepeatedOutInCommand to 0 with new SequencerRepeatedOutIn command.

◆ SequencerSetI2CParameters()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
SequencerSetI2CParameters ( const unsigned int & Sequencer,
                            uint8_t              I2C_0_Destination,
                            uint8_t              I2C_delay_start_stop,
                            uint8_t              I2C_delay_data_setup,
                            uint8_t              I2C_delay_clock_high,
                            uint8_t              I2C_delay_clock_low,
                            uint8_t              I2C_delay_pause_before_read )
```

Sets I2C parameters for the sequencer.

Parameters

Sequencer	the sequencer to use.
I2C_0_Destination	destination or routing value for I2C bus. Currently unused.
I2C_delay_start_stop	start/stop timing delay in sequencer timing units, i.e. 10ns for the sequencer internal 100MHz clock.
I2C_delay_data_setup	data setup timing delay in sequencer timing units.
I2C_delay_clock_high	clock-high timing delay in sequencer timing units.
I2C_delay_clock_low	clock-low timing delay in sequencer timing units.
I2C_delay_pause_before_read	pause before read phase in sequencer timing units.

◆ SequencerSetLoopCount()

API_EXPORT ERROR_CODE_TYPE CLA_FN SequencerSetLoopCount (const unsigned int & Sequencer,
unsigned int loop_count)

Sets the loop count for a sequencer.

Parameters

Sequencer	the sequencer to use.
loop_count	the loop count to set.

◆ SequencerSetSPIMode()

API_EXPORT ERROR_CODE_TYPE CLA_FN SequencerSetSPIMode (const unsigned int & Sequencer,
uint8_t SPI_mode)

Sets the SPI mode for the sequencer.

Parameters

Sequencer the sequencer to use.

SPI_mode SPI mode number, normally 0..3 for CPOL/CPHA selection. Mode CPOL CPHA Sample edge 0 0 0 rising edge 1 0 1 falling edge 2 1 0 falling edge 3 1 1 rising edge

◆ SequencerSetSPITiming()

API_EXPORT ERROR_CODE_TYPE CLA_FN
SequencerSetSPITiming (const unsigned int & Sequencer,
uint16_t SPI_delay_CS_low_start_wait,
uint16_t SPI_delay_write,
uint16_t SPI_delay_pause_before_read,
uint16_t SPI_delay_read,
uint16_t SPI_delay_CS_low_end_wait)

Loads SPI timing parameters into the sequencer.

Parameters

Sequencer	the sequencer to use.
SPI_delay_CS_low_start_wait	delay after chip-select goes low before writing starts, in sequencer timing units, i.e. 10ns for the sequencer internal 100MHz clock.
SPI_delay_write	delay used while writing SPI bits, in sequencer timing units.
SPI_delay_pause_before_read	delay between write and read phases, in sequencer timing units.
SPI_delay_read	delay used while reading SPI bits, in sequencer timing units.
SPI_delay_CS_low_end_wait	delay before chip-select is released, in sequencer timing units.

◆ SequencerSetTimeDebtGuard_in_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SequencerSetTimeDebtGuard_in_ms (const unsigned int & Sequencer,
const double & MaxTimeDebt_in_ms)

Sets the time debt guard for a specific sequencer.

Parameters

Sequencer	the sequencer to use.
MaxTimeDebt_in_ms	the maximum time debt in ms.

◆ SequencerStartAnalogInAcquisition()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SequencerStartAnalogInAcquisition (const unsigned int & Sequencer,
const uint8_t & analog_in_type,
const uint8_t & SPI_CS,
const uint8_t & ChannelNumber,
const uint32_t & NumberOfDataPoints,
const double & DelayBetweenDataPoints_in_ms)

Start analog acquisition on the specified channel. This function places a commandto for the FPGA in the sequencer buffer. The analog in acquisition will start when this command is executed by the FPGA. Only one analog in acquisition can hapen at each moment. A new one can start right after the previous one is finished. The data will be returned at the end of a sequence when using CLA_WaitTillEndOfSequenceThenGetInputData.

Parameters

Sequencer	the sequencer to use.
analog_in_type	Analog in board type. 0: AQuRA MCP3208 analog in board; 1: MCP3208 12-bit ADC on SerialPortBoard; 2: ADS1256 24-bit ADC.
SPI_CS	SPI chip-select setting to use. This is the 3-bit CS bit-pattern on the rack backplane. A 3-to-8 demultiplexer is used on the serial port board to create 8 CS lines. If a board only needs 3 or less CS lines, these lines can be used directly (in that case: make sure to only pull one line low at a time).
ChannelNumber	the channel number to use.
NumberOfDataPoints	the number of data points to acquire.
DelayBetweenDataPoints_in_ms	the delay between data points in ms.

◆ SequencerSwitchDebugLED()

API_EXPORT ERROR_CODE_TYPE CLA_FN SequencerSwitchDebugLED (const unsigned int & Sequencer,
unsigned int OnOff)

Switches the debug LED of the FPGA on or off.

Parameters

Sequencer the sequencer to use.

OnOff true to switch on the LED, false to switch it off.

Returns

See return convention above.

◆ SequencerTransmitI2C()

API_EXPORT ERROR_CODE_TYPE CLA_FN SequencerTransmitI2C (const unsigned int & Sequencer,
uint8_t I2C_port,
uint8_t I2C_length_out,
uint8_t I2C_length_in,
uint8_t * data_out)

Writes an I2C command to the sequencer.

Parameters

Sequencer the sequencer to use.

I2C_port I2C port index on the sequencer. 1 is the I2C port exposed by the serial port board and used by external hardware. 0 is the I2C port connected to the backplane multiplexer, reading out board EEPROMS and addresses.

I2C_length_out number of bytes to transmit from data_out.

I2C_length_in number of bytes to read back after the write phase.

data_out bytes to transmit. The buffer must contain at least I2C_length_out bytes.

◆ SequencerTransmitSPI()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
SequencerTransmitSPI ( const unsigned int & Sequencer,
                       uint8_t               chip_select,
                       uint16_t              number_of_bits_out,
                       const uint8_t *       data_out,
                       uint8_t               number_of_bits_in,
                       bool                  start_now )
```

Writes an SPI transmit command to the sequencer.

Parameters

Sequencer	the sequencer to use. (SPI port 1 will be used, which is the port used by existing control hardware and exposed by the serial port board to the user.)
chip_select	SPI chip-select line or encoded chip-select value. This is the 3-bit CS bit-pattern on the rack backplane. A 3-to-8 demultiplexer is used on the serial port board to create 8 CS lines. If a board only needs 3 or less CS lines, these lines can be used directly (in that case: make sure to only pull one line low at a time).
number_of_bits_out	number of bits to transmit from data_out.
data_out	bit payload to transmit. The buffer must contain enough bytes for number_of_bits_out.
number_of_bits_in	number of bits to read back after the write phase.
start_now	true to start the SPI transfer immediately when the command is executed; false to configure it for SequencerRepeatedOutIn without immediate start.

◆ SequencerWriteInputMemory()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SequencerWriteInputMemory (const unsigned int & Sequencer,
unsigned long input_buf_mem_data,
bool write_next_address = 1,
unsigned long input_buf_mem_address = 0)

Writes a value to the input memory of the sequencer. This is useful to mark the start or end of a data acquisition, or to mark the type of experimental run in the full fledged version of the ControlAPI, which can cycle sequences automatically in the background. To execute this function, a command for the FPGA must be placed in the sequencer buffer.

Parameters

Sequencer	the sequencer to use.
input_buf_mem_data	the data to write to the input memory.
write_next_address	true to write the next address, false to write the address given.
input_buf_mem_address	the address to write to, if write_next_address is false.

◆ SequencerWriteSystemTimeToInputMemory()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SequencerWriteSystemTimeToInputMemory (const unsigned int & Sequencer)

Writes the current FPGA clock ticks to the input memory of the sequencer.

Parameters

Sequencer	the sequencer to use.

◆ SetAttenuation()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetAttenuation (const unsigned int & Sequencer,
const unsigned int & Address,
double Attenuation)

Sets the attenuation of a DDS (for now a AD9854 DDS).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the attenuation for.

Attenuation the attenuation to set in dB (-xxx ... 0).

Returns

See return convention above.

◆ SetClearACC1()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetClearACC1 (const unsigned int & Sequencer,
const unsigned int & Address,
bool OnOff)

Clears the ACC1 bit of a DDS (for now a AD9854 DDS).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the ACC1 bit for.

OnOff true to set the ACC1 bit, false to clear it.

Returns

See return convention above.

◆ SetDigitalOutput()

```
API_EXPORT ERROR_CODE_TYPE CLA_FN SetDigitalOutput ( const unsigned int & Sequencer,
                                                     const unsigned int & Address,
                                                     uint8_t              BitNr,
                                                     bool                 OnOff )
```

Sets a digital output to high or low.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the digital output for.
BitNr	the bit number of the digital output to set.
OnOff	true to set the output to high, false to set it to low.

◆ SetFrequency()

```
API_EXPORT ERROR_CODE_TYPE CLA_FN SetFrequency ( const unsigned int & Sequencer,
                                                 const unsigned int & Address,
                                                 double               Frequency )
```

Sets the frequency of a DDS.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the frequency for.
Frequency	the frequency to set.

Returns

See return convention above.

◆ SetFrequencyOfChannel()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetFrequencyOfChannel (const unsigned int & Sequencer,
const unsigned int & Address,
uint8_t channel,
double Frequency)

Sets the frequency of a multi channel DDS (for now the AD9959).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the frequency for.

channel the channel number to set the frequency for.

Frequency the frequency to set.

Returns

See return convention above.

◆ SetFrequencyTuningWord()

API_EXPORT ERROR_CODE_TYPE CLA_FN
SetFrequencyTuningWord (const unsigned int & Sequencer,
const unsigned int & Address,
uint64_t FrequencyTuningWord)

Sets the frequency tuning word of a DDS.

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the frequency tuning word for.

FrequencyTuningWord the frequency tuning word to set.

Returns

See return convention above.

◆ SetFrequencyTuningWordOfChannel()

API_EXPORT ERROR_CODE_TYPE CLA_FN
SetFrequencyTuningWordOfChannel (const unsigned int & Sequencer,
const unsigned int & Address,
uint8_t channel,
uint64_t FrequencyTuningWord)

Sets the frequency tuning word of a multi channel DDS (for now the AD9959).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the frequency tuning word for.
channel	the channel number to set the frequency tuning word for.
FrequencyTuningWord	the frequency tuning word to set.

Returns

See return convention above.

◆ SetFSKBit()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetFSKBit (const unsigned int & Sequencer,
const unsigned int & Address,
bool OnOff)

Sets the FSK bit of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the FSK bit for.
OnOff	true to set the FSK bit, false to clear it.

Returns

See return convention above.

◆ SetFSKMode()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetFSKMode (const unsigned int & Sequencer,
const unsigned int & Address,
uint8_t mode)

Sets the FKS mode of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the FKS mode for.
mode	the FKS mode to set (0..4).

Returns

See return convention above.

◆ SetIOUpdateEnabled()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetIOUpdateEnabled (const unsigned int & Sequencer,
const unsigned int & Address,
bool IOUpdateEnabled)

Enables or disables automatic IO update after commands sent to a DDS device that supports IO update control.

Parameters

Sequencer	the sequencer to use.
Address	the address of the DDS device.
IOUpdateEnabled	true to enable automatic IO update after each SPI command, false to leave IO update under explicit/manual control.

Returns

See return convention above.

◆ SetModulationFrequency()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetModulationFrequency (const unsigned int & Sequencer,
const unsigned int & Address,
double Frequency)

Sets the modulation frequency of a DDS (for now a AD9854 DDS).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the modulation frequency for.

Frequency the frequency to set.

Returns

See return convention above.

◆ SetPeriodicTrigger_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN
SetPeriodicTrigger_ms (double PeriodicTriggerPeriod_in_ms,
double PeriodicTriggerAllowedWaitTime_in_ms)

Set the periodic trigger timing used by the master sequencer.

Parameters

PeriodicTriggerPeriod_in_ms the period of the periodic trigger in ms.

PeriodicTriggerAllowedWaitTime_in_ms the maximum time in ms the master sequencer may wait for the next periodic trigger before reporting an error.

Returns

See return convention above.

◆ SetPhaseOfChannel()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetPhaseOfChannel (const unsigned int & Sequencer,
const unsigned int & Address,
uint8_t channel,
double Phase)

Sets the phase of a multi channel DDS (for now the AD9959).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the phase for.

channel the channel number to set the phase for.

Phase the phase to set.

Returns

See return convention above.

◆ SetPower()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetPower (const unsigned int & Sequencer,
const unsigned int & Address,
double Power)

Sets the power of a DDS (for now a AD9854 DDS).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the power for.

Power the power to set in % (0...100).

Returns

See return convention above.

◆ SetPowerOfChannel()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetPowerOfChannel (const unsigned int & Sequencer,
const unsigned int & Address,
uint8_t channel,
double Power)

Sets the power of a multi channel DDS (for now the AD9959).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the power for.

channel the channel number to set the power for.

Power the power to set in percent (0...100).

Returns

See return convention above.

◆ SetPSOptions()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetPSOptions (uint8_t options)

Set PS option flags on the master sequencer controller.

Parameters

options option bitmask passed through to the controller firmware.

Returns

See return convention above.

◆ SetRampRateClock()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetRampRateClock (const unsigned int & Sequencer,
const unsigned int & Address,
uint8_t rate)

Sets the ramp rate clock of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the ramp rate clock for.
rate	the ramp rate clock to set (1...1048576).

Returns

See return convention above.

◆ SetRegister()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetRegister (const unsigned int & Sequencer,
const unsigned int & Address,
const unsigned int & SubAddress,
const uint8_t * Data,
const unsigned long & DataLength_in_bit,
const uint8_t & StartBit = 0)

This function sets a register for a device on the sequencer. What this means depends on the device type. This function gives us an easy way to add new functionality to a device without having to programm new DLL functions. SetRegister maps to the register map given in the device's datasheet. Note: this is not currently implemented in the API.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the value for.
SubAddress	the subaddress of the device to set the value for.
Data	the data to set.
DataLength_in_bit	the length of the data in bits.
StartBit	the start bit of the data to set.

Returns

See return convention above.

◆ SetRegisterSerialDevice()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
SetRegisterSerialDevice ( const unsigned int &  Sequencer,
                          const unsigned int &  Address,
                          const unsigned int &  SubAddress,
                          const uint8_t *       Data,
                          const unsigned long & DataLength_in_bit,
                          const uint8_t &       StartBit = 0 )
```

As SetRegister, but for serial devices.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the value for.
SubAddress	the subaddress of the device to set the value for.
Data	the data to set.
DataLength_in_bit	the length of the data in bits.
StartBit	the start bit of the data to set.

◆ SetSequencer_PL_to_PS_command()

API_EXPORT ERROR_CODE_TYPE CLA_FN

```
SetSequencer_PL_to_PS_command ( const unsigned int & Sequencer,
                                uint8_t              PL_to_PS_command )
```

Sets the sequencer PL-to-PS command byte.

Parameters

Sequencer	the sequencer to use.
PL_to_PS_command	the 8-bit PL-to-PS command.

◆ SetSequencerDigitalOut()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetSequencerDigitalOut (const unsigned int & Sequencer,
uint8_t dig_out_pattern)

Sets the sequencer digital output pattern.

Parameters

Sequencer the sequencer to use.

dig_out_pattern the 8-bit digital output pattern.

◆ SetStartFrequency()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetStartFrequency (const unsigned int & Sequencer,
const unsigned int & Address,
double Frequency)

Sets the start frequency of a DDS (for now a AD9854 DDS).

Parameters

Sequencer the sequencer to use.

Address the address of the device to set the start frequency for.

Frequency the frequency to set.

Returns

See return convention above.

◆ SetStartFrequencyTuningWord()

API_EXPORT ERROR_CODE_TYPE CLA_FN
SetStartFrequencyTuningWord (const unsigned int & Sequencer,
const unsigned int & Address,
uint64_t FrequencyTuningWord)

Sets the start frequency tuning word of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the start frequency tuning word for.
FrequencyTuningWord	the frequency tuning word to set.

Returns

See return convention above.

◆ SetStopFrequency()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetStopFrequency (const unsigned int & Sequencer,
const unsigned int & Address,
double Frequency)

Sets the stop frequency of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the stop frequency for.
Frequency	the frequency to set.

Returns

See return convention above.

◆ SetStopFrequencyTuningWord()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SetStopFrequencyTuningWord (const unsigned int & Sequencer,
const unsigned int & Address,
uint64_t FrequencyTuningWord)

Sets the stop frequency tuning word of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the stop frequency tuning word for.
FrequencyTuningWord	the frequency tuning word to set.

Returns

See return convention above.

◆ SetTimeDebtGuard_in_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN

SetTimeDebtGuard_in_ms (const double & MaxTimeDebt_in_ms)

Sets a guard time. If sequencer commands make the time advance more than the guard time beyond what's allowed by Wait_ms, an error will be recorded (check with **DidErrorOccur()** if that happened) or thrown.

Parameters

MaxTimeDebt_in_ms	maximum time debt in ms. If the sequencer commands make the time advance more than this, the sequencer will stop and wait for the next command.

Returns

See return convention above.

◆ SetTriangleBit()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetTriangleBit (const unsigned int & Sequencer,
const unsigned int & Address,
bool OnOff)

Sets the triangle bit of a DDS (for now a AD9854 DDS).

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the triangle bit for.
OnOff	true to set the triangle bit, false to clear it.

Returns

See return convention above.

◆ SetValue()

API_EXPORT ERROR_CODE_TYPE CLA_FN SetValue (const unsigned int & Sequencer,
const unsigned int & Address,
const unsigned int & SubAddress,
const uint8_t * Data,
const unsigned long & DataLength_in_bit,
const uint8_t & StartBit = 0)

This function sets a value for a device on the sequencer. What this means depends on the device type. This function gives us an easy way to add new functionality to a device without having to programm new DLL functions. In contrast to SetRegister, SetValue can execute more complex operations, such as calculating a DDS frequency tuning word from the given frequency and then programming that. There can be several SetValue functions for the same SetRegister function. There can be SetValue functions that set some parameter, or that trigger some action, not even necessarily related to the registers of the device.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the value for.
SubAddress	the subaddress of the device to set the value for.
Data	the data to set.
DataLength_in_bit	the length of the data in bits.
StartBit	the start bit of the data to set.

Returns

See return convention above.

◆ SetValueSerialDevice()

```
API_EXPORT ERROR_CODE_TYPE CLA_FN SetValueSerialDevice ( const unsigned int &      Sequencer,
                                                         const unsigned int &      Address,
                                                         const unsigned int &      SubAddress,
                                                         const uint8_t *           Data,
                                                         const unsigned long &     DataLength_in_bit,
                                                         const uint8_t &           StartBit = 0 )
```

As SetValue, but for serial devices.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the value for.
SubAddress	the subaddress of the device to set the value for.
Data	the data to set.
DataLength_in_bit	the length of the data in bits.
StartBit	the start bit of the data to set.

Returns

See return convention above.

◆ SetVoltage()

```
API_EXPORT ERROR_CODE_TYPE CLA_FN SetVoltage ( const unsigned int & Sequencer,
                                               const unsigned int & Address,
                                               double               Voltage )
```

Sets the voltage of an analog output device.

Parameters

Sequencer	the sequencer to use.
Address	the address of the device to set the voltage for.
Voltage	the voltage to set.

◆ StartAssemblingCPUCommandSequence()

API_EXPORT ERROR_CODE_TYPE CLA_FN StartAssemblingCPUCommandSequence ( )

Start assembling a CPU command sequence on the master sequencer.

Returns

See return convention above.

◆ StartAssemblingNextSequence()

API_EXPORT void **CLA_FN** StartAssemblingNextSequence ( )

Starts assembling the second or higher sequence in the sequence buffer. Clears any previous sequence. Must be called before adding commands to the sequence.

Returns

See return convention above.

◆ StartAssemblingSequence()

API_EXPORT void **CLA_FN** StartAssemblingSequence ( )

Starts assembling the first sequence in the sequency buffer. Clears any previous sequence. Must be called before adding commands to the sequence.

Returns

See return convention above.

◆ StopCPUCommandSequence()

API_EXPORT ERROR_CODE_TYPE CLA_FN StopCPUCommandSequence ( )

Stop CPU command sequence at next programmed stop point.

Returns

See return convention above.

◆ SwitchDebugMode()

API_EXPORT void **CLA_FN** SwitchDebugMode (bool OnOff,
const char * FileName)

Switches FPGA debug mode on. In debug mode, the FPGA sequencers display more information on their USB-UART port, being slowed down a bit by that. To write a human-readable sequence file, pass a filename to SendSequence instead.

Parameters

OnOff true to switch on debug mode, false to switch it off.

FileName retained for compatibility and ignored.

◆ SwitchSequencerBuzzer()

API_EXPORT ERROR_CODE_TYPE CLA_FN SwitchSequencerBuzzer (const unsigned int & Sequencer,
bool OnOff)

Switches the sequencer buzzer on or off.

Parameters

Sequencer the sequencer to use.

OnOff true to switch on the buzzer, false to switch it off.

◆ TransmitI2CPort()

API_EXPORT ERROR_CODE_TYPE CLA_FN TransmitI2CPort (uint8_t I2C_port,
uint8_t I2C_destination,
uint8_t I2C_address,
uint16_t send_length,
uint8_t * send_data,
uint16_t receive_length,
uint8_t * receive_data,
uint32_t I2C_clock_frequency_in_Hz,
bool & I2C_success,
bool fail_silently)

Read data from an I2C port.

Parameters

I2C_port	the I2C port to read from.
I2C_destination	If I2C_port == 0, 0 means port connected to rack configuration I2C channel, 1 means port connected to z-turn I2C.
I2C_address	the I2C address to read from.
send_length	the length of the data to send in bytes.
send_data	the data to send.
receive_length	the length of the data to receive in bytes.
receive_data	the buffer to store the received data.
I2C_clock_frequency_in_Hz	the I2C clock frequency in Hz.
I2C_success	output flag set by the sequencer to indicate whether the I2C transaction was acknowledged and completed.
fail_silently	true to suppress API error reporting for an I2C failure while still updating I2C_success.

Returns

See return convention above.

◆ TransmitOnlyDifferenceBetweenCommandSequenceIfPossible()

API_EXPORT void **CLA_FN** TransmitOnlyDifferenceBetweenCommandSequenceIfPossible (bool OnOff)

Transmits only changes of sequence over TCP/IP to sequencer in order to safe time. Once a sequence is in the sequencer's memory, the next sequence is compared to the one in memory and only the changes are transmitted over TCP/IP, if this results in less transmission data.

Parameters

OnOff true to switch on debug mode, false to switch it off.

◆ UseEdgeTriggeredLatches()

API_EXPORT ERROR_CODE_TYPE CLA_FN
UseEdgeTriggeredLatches (const unsigned int & Sequencer,
bool UseEdgeTriggeredLatches)

Configures the sequencer to use edge-triggered latches.

Parameters

Sequencer the sequencer to use.

UseEdgeTriggeredLatches true to enable edge-triggered latches, false to use the default latch timing.

◆ Wait_ms()

API_EXPORT ERROR_CODE_TYPE CLA_FN Wait_ms (double time_in_ms)

Wait for a given time in ms.

Parameters

time_in_ms the time to wait in ms.

Returns

See return convention above.

◆ WaitTillEndOfSequenceThenGetInputData()

API_EXPORT ERROR_CODE_TYPE CLA_FN
WaitTillEndOfSequenceThenGetInputData (uint8_t *& buffer,
unsigned long & buffer_length,
unsigned long & EndTimeOfCycle,
double timeout_in_s)

Waits until the sequence is finished, then gets the input data from the FPGA sequencer.

Parameters

buffer	pointer to the input data buffer. Don't delete this buffer, it is managed by the API.
buffer_length	length of the input data buffer in bytes.
EndTimeOfCycle	returns the end time of the cycle in ms.
timeout_in_s	timeout in seconds. If timeout_in_s > 0.001 and the sequence is not finished within this time, the function returns false or throws an exception (depending on mode selected mode when compiling API).

◆ WaitTillFinished()

API_EXPORT ERROR_CODE_TYPE CLA_FN WaitTillFinished (double timeout_in_s = `0`)

Waits until the sequence is finished.

Parameters

timeout_in_s	timeout in seconds. If timeout_in_s > 0.001 and the sequence is not finished within this time, the function returns false or throws an exception (depending on mode selected mode when compiling API).

Returns

See return convention above.

◆ WriteConfigAddress()

API_EXPORT ERROR_CODE_TYPE CLA_FN WriteConfigAddress (uint8_t SequencerID,
uint8_t RackNr,
uint8_t SlotNr,
uint8_t address)

Write the configuration address byte of a rack slot.

Parameters

SequencerID	the sequencer to use.
RackNr	the rack number.
SlotNr	the slot number within the rack.
address	the address byte to write.

Returns

See return convention above.

◆ WriteConfigEEPROM()

API_EXPORT ERROR_CODE_TYPE CLA_FN WriteConfigEEPROM (uint8_t SequencerID,
uint8_t RackNr,
uint8_t SlotNr,
const char * data,
size_t length)

Write configuration data to the EEPROM of a rack slot.

Parameters

SequencerID	the sequencer to use.
RackNr	the rack number.
SlotNr	the slot number within the rack.
data	the configuration payload to write.
length	the payload length in bytes.

Returns

See return convention above.

Variable Documentation

◆ ErrorListWrapAround

bool ErrorListWrapAround extern

◆ LastError

std::string LastError[**MaxLastError**] extern

◆ LastErrorIndex

int LastErrorIndex extern

◆ MaxLastError

int MaxLastError = 10 constexpr